



SaaS de Migration AFP

Analyse de Faisabilite Technique

Date : 13/04/2026

Document confidentiel

Ce document presente une analyse technique detaillee de la faisabilite d'un SaaS permettant la migration automatisee des flux documentaires IBM AFP (Advanced Function Presentation) vers des formats modernes (PDF/PDF-A). Il couvre l'architecture AFP, les ressources open source disponibles, les defis techniques majeurs, l'architecture proposee, et une estimation des efforts de developpement.

Table des matieres

1. L'architecture AFP : ce qu'il faut parser
2. Les bibliotheques open source existantes
3. Defi n°1 : les polices
4. Defi n°2 : les ressources externes
5. Defi n°3 : les cas degradés
6. La validation de fidelite
7. Architecture technique proposee
8. Choix technologiques
9. Estimation des efforts
10. Synthese de faisabilite

1. L'architecture AFP : ce qu'il faut parser

L'AFP n'est pas un simple format de fichier comme PDF. C'est un ensemble d'architectures imbriquées, dont la colonne vertébrale est la spécification MO:DCA (Mixed Object Document Content Architecture). Elle se décompose en sous-architectures spécialisées :

Architecture	Fonction	Description
MO:DCA	Structure documentaire	Documents, pages, groupes de ressources, enveloppes
PTOCA	Texte	Positionnement des caractères, polices, orientation
GOCA	Graphiques vectoriels	Lignes, courbes, arcs, zones remplies
IOCA	Images raster	Bi-level, greyscale, couleur, compression
BCOCA	Codes-barres	1D, 2D, QR Codes (depuis 2023)
FOCA	Métriques de polices	Dimensions, espacement, mapping caractères
CMOCA	Gestion couleurs	Profils ICC, espaces colorimétriques
MOCA	Metadonnées	XMP, metadonnées d'archivage (ISO 18565)

L'AFP Consortium a publié en juin 2023 la 11e édition de la référence MO:DCA et la 13e édition de l'IPDS, avec de nouvelles fonctionnalités (Setup Names, QR Codes avec image, texte chiffre PTOCA). L'architecture est donc encore vivante et activement mise à jour.

Verdict technique : Le parsing est complexe mais bien documenté. Les spécifications MO:DCA, PTOCA, GOCA, IOCA, BCOCA, FOCA sont gratuites et téléchargeables en PDF depuis le site de l'AFP Consortium (afpconsortium.org). Pas de reverse engineering nécessaire.

2. Les bibliothèques open source existantes

Plusieurs parseurs AFP open source existent, principalement en Java. Aucun parseur AFP mature n'existe en Python à ce jour.

Alpheus AFP Parser (Java, GPL v3)

Implémentation complète : chaque Structured Field, Repeating Group et Triplet est entièrement implémenté en tant que classe Java. Couvre toutes les spécifications AFP (MO:DCA, BCOCA, CMOCA, FOCA, GOCA, IOCA, PTOCA), sans aucune dépendance externe. C'est le parseur open source le plus complet disponible. Licence GPL v3.

afpbox (Java, open source)

Bibliothèque Java de parsing des flux AFP/MO:DCA, sans dépendance externe, implémentée en s'appuyant sur les références officielles. Projet encore en work in progress avec une couverture partielle des Structured Fields.

afplib (Java, EMF)

API bas niveau pour transformer les fichiers AFP (structured fields, triplets) en objets Java et vice-versa, construite sur Eclipse Modeling Framework. L'auteur a construit un produit commercial (AFPWorx) par-dessus.

Apache FOP (Java, Apache License)

Le renderer AFP de FOP crée des documents conformes à l'architecture MO:DCA d'IBM, mais il s'agit d'un générateur AFP (XSL-FO vers AFP), pas d'un parseur dans le sens inverse. Support limité : pas de clipping, polices restreintes.

Verdict technique : Alpheus est la base la plus solide pour démarrer, mais sa licence GPL v3 est contraignante pour un SaaS commercial. Deux options : négocier une licence commerciale avec l'auteur (Rudolf Fiala), ou développer un parseur propriétaire en s'appuyant sur les spécifications publiques de l'AFP Consortium (choix stratégique le plus défendable à long terme).

3. Defi n°1 : les polices

C'est le defi technique le plus critique de toute la chaine de conversion. Les polices AFP se declinent en plusieurs types, chacun posant des problemes specifiques :

Type	Format	Difficulte de conversion
Polices raster IBM	Bitmap, liees a une resolution (240/300/600 dpi), character sets + code pages EBCDIC	ELEVEE
Polices outline AFP (FOCA)	Vectérielles, format propriétaire IBM	MOYENNE
TrueType/OpenType embarquées	Format standard, encapsulees dans conteneurs AFP	FAIBLE
Polices custom client	Creees/modifiees sur des decennies, non repertoriees dans les tables standard	TRES ELEVEE

Le processus de conversion necessite de mapper les polices AFP vers des polices TrueType ou Type 1 compatibles PDF, avec des fichiers de metriques personnalises pour controler le positionnement individuel de chaque caractere et assurer l'alignement du texte. Pour les polices custom, il faut manuellement editer les fichiers de definition — c'est la procedure standard actuelle chez tous les concurrents.

Approche proposee : mapping assiste par IA

Constituer une base de donnees de mapping couvrant les polices IBM standard (quelques centaines), puis utiliser de l'IA/vision par ordinateur pour automatiser le mapping des polices custom, en comparant le rendu visuel des glyphes AFP avec les glyphes de polices TrueType/OpenType candidates. C'est precisement la ou l'IA pourrait apporter un avantage competitif reel par rapport aux solutions existantes.

Verdict technique : C'est le point le plus dur du projet. Les polices standard sont gereables ; les polices custom constituent le risque technique majeur. L'approche IA pour le mapping automatique est un differentiateur potentiel.

4. Defi n°2 : les ressources externes

Il n'est pas rare que les ressources AFP (polices, images, overlays, form definitions, page definitions, page segments, etc.) soient separees du fichier d'impression pour reduire la taille des fichiers. Un flux AFP peut référencer des ressources stockees dans des bibliotheques sur le mainframe (FONTLIB, OVLYLIB, PSFLIB) sans les embarquer.

Quand les ressources sont manquantes, le rendu AFP est degrade : polices incorrectes, lignes de texte qui se chevauchent, images absentes. Pour la migration, cela signifie qu'il ne suffit pas d'ingerer un fichier AFP : il faut aussi collecter l'ensemble des ressources referencees depuis

l'environnement mainframe du client.

Verdict technique : L'outil devra inclure un agent collecteur a deployer cote client (sur IBM i ou z/OS) qui inventorie et exporte toutes les ressources referencees. Pre-requis non negociable pour garantir la fidelite.

5. Defi n°3 : les cas degrades

Certains fichiers AFP ne sont pas faciles a retraiter. Les cas problematiques incluent :

- â■ Pages entieres sous forme d'images fax, sans caracteres ni polices referencees (necessite OCR)
- â■ Fichiers PDF embarques dans des conteneurs d'objets AFP
- â■ Fichiers mal structures : polices dupliquees, elements graphiques redondants, ressources manquantes
- â■ Formats legacy (FS10, bi-level) cotoyes avec des formats modernes (couleur, FS45) dans le meme flux
- â■ Donnees DBCS (double-byte character sets) pour langues asiatiques avec composition automatique des voyelles

Verdict technique : Prevoir une couche de normalisation en amont et une classification automatique par niveau de complexite. Routing automatique pour les cas simples, revue humaine assistee pour les cas complexes. Viser 80% automatique, 20% assiste.

6. La validation de fidelite

C'est ce qui fera la difference entre un convertisseur amateur et un produit enterprise-grade. La validation doit operer a trois niveaux :

Comparaison visuelle

Rasteriser l'AFP original (via emulateur IPDS) et le PDF genere, puis calculer un score de similarite pixel par pixel. Outils : ImageMagick, OpenCV. Seuil d'acceptation configurable par le client.

Validation structurelle

Verifier que tous les elements (texte, images, codes-barres, overlays) sont presents et correctement positionnes. Comptage d'objets, verification des coordonnees, integrite des liens.

Validation des metadonnees

Verification des TLE (Tag Logical Elements), NOP records, index documentaires. Essentiel pour les workflows d'archivage et de recherche documentaire.

Verdict technique : Faisable avec des outils existants (ImageMagick, OpenCV). Le defi principal est d'obtenir un rendu de reference AFP fiable sans imprimante IPDS physique. Il faudra investir dans un environnement de reference (emulateur IPDS ou partenariat IBM).

7. Architecture technique proposee

Vue d'ensemble du pipeline

Couche	Composant	Technologies
Client	Agent Collecteur	Script IBM i (RPGLE/CL) ou z/OS (JCL/REXX), export SFTP chiffre
Ingestion	Normalisation AFP	Parseur Java custom, deduplication, resolution ressources
Analyse	Inventaire + Mapping	Python (IA), base de mapping polices, classification complexite
Conversion	AFP vers PDF	Java (parseur) + Apache PDFBox (generation PDF), representation intermediaire XML
Validation	QA automatisee	OpenCV, ImageMagick, scoring confiance, routing auto/manuel
Dashboard	Interface utilisateur	React/TypeScript, suivi progression, gestion exceptions

Flux de traitement

1. L'agent collecteur cote client exporte les flux AFP et toutes les ressources referencees vers la plateforme via un canal securise.
2. La couche d'ingestion parse les fichiers AFP, resout les references aux ressources externes, normalise les flux et classeifie chaque document par niveau de complexite.
3. L'analyse automatique identifie les polices, overlays, form definitions et propose un mapping vers les equivalents modernes. L'IA assiste le mapping des polices custom non standard.
4. Le moteur de conversion transforme les flux AFP en une representation intermediaire (XML/JSON structuree), puis genere les PDF avec toutes les metadonnees preservees.
5. Le systeme de QA valide automatiquement chaque document converti (comparaison visuelle, structurelle, metadonnees) et attribue un score de confiance. Les documents sous le seuil sont routes vers une revue humaine assistee.

8. Choix technologiques

Composant	Langage	Justification
Parseur AFP / Conversion	Java	Toutes les libs OSS sont en Java. Ecosysteme enterprise familier pour les clients BFSI. Apache PDFBox pour la sortie PDF.
Analyse IA / Mapping	Python	Ecosysteme ML/CV mature (PyTorch, OpenCV, scikit-learn). Ideal pour le mapping polices par vision.
Agent collecteur	RPGLE / CL / JCL	Langages natifs IBM i / z/OS. Necessary pour acceder aux bibliotheques de ressources mainframe.
Dashboard	React / TypeScript	Stack moderne, performante. Coherent avec les competences existantes de l'equipe.
Backend API	Java (Spring Boot) ou Node.js	Orchestration du pipeline, gestion des jobs, API REST. Spring Boot si Java dominant dans l'equipe.

L'alternative Rust pour le parseur AFP offrirait de meilleures performances de parsing (flux AFP de plusieurs gigaoctets), mais impliquerait de repartir de zero sans capitaliser sur l'existant open source Java. A considerer uniquement si les performances deviennent un goulot d'etranglement.

9. Estimation des efforts

Estimation prudente pour un MVP credible, avec une equipe de 3-4 developpeurs :

Composant	Duree	Ressources	Prerequis
Parseur AFP (80% des cas)	3-4 mois	2 dev Java senior	Specs AFP Consortium
Moteur mapping polices	2-3 mois	1 dev + 1 ML eng.	Base polices IBM, dataset training
Pipeline conversion AFP vers PDF	2-3 mois	2 dev Java	Apache PDFBox, parseur fonctionnel
Systeme QA automatise	1-2 mois	1 dev Python	OpenCV, env. reference AFP
Agent collecteur (IBM i / z/OS)	1-2 mois	1 expert mainframe	Acces environnement IBM
Dashboard + API	1-2 mois	1 dev fullstack	Pipeline fonctionnel
Integration + tests clients	2-3 mois	Equipe complete	Client pilote avec donnees reelles

Total estime : 12-18 mois pour un MVP credible. Le recrutement d'au moins un expert AFP (probablement semi-retraite de l'ecosysteme IBM) est quasi indispensable pour eviter des mois de tatonnement sur les cas limites.

10. Synthèse de faisabilité

Brique technique	Difficulté	Ressources existantes	Risque
Parsing MO:DCA	Moyenne	Specs publiques + parseurs OSS	Modere
Mapping polices standard	Moyenne	Tables IBM connues	Faible
Mapping polices custom	ELEVEE	Quasi rien	ELEVEE
Ressources externes	Moyenne	Bien documente	Modere
Conversion vers PDF	Moyenne	Apache PDFBox, iText	Faible
Validation fidelite	Moy.-haute	OpenCV, ImageMagick	Modere
Cas degradés (fax, PDF emb.)	ELEVEE	OCR (Tesseract)	ELEVEE
Agent collecteur mainframe	Moyenne	APIs IBM i documentees	Modere

Conclusion

Le projet est techniquement faisable mais exige des competences rares et un investissement initial consequent. Le plus grand risque est le "long tail" des cas particuliers que chaque entreprise accumule depuis 30-40 ans.

La cible realiste pour un MVP est une couverture de 80% des cas en traitement automatique, avec un workflow assiste pour les 20% restants. Le timing est favorable : les experts partent a la retraite, IBM se desengage des outils AFP, les reglementations (DORA) poussent a la modernisation, et les grands acteurs (AWS, IBM) investissent massivement dans la modernisation mainframe, ce qui eduque le marche.

Les facteurs cles de succes sont : le recrutement d'au moins un expert AFP senior, l'obtention d'un client pilote avec des donnees reelles des les premiers mois, et le choix strategique entre exploitation des bibliotheques open source existantes (Alpheus, GPL v3) ou developpement d'un parseur propriétaire (investissement plus lourd mais independance totale).

Sources et references

â─ AFP Consortium - Specifications MO:DCA, PTOCA, GOCA, IOCA, BCOCA, FOCA (afpconsortium.org)

â─ ISO 18565:2015 - Document management, AFP/Archive

â─ ISO 22550:2021 - Document management, AFP interchange for PDF

-
- â─ Alpheus AFP Parser - github.com/afpdev/alpheusafpparser
 - â─ afpbox - github.com/michaelknigge/afpbox
 - â─ afplib - github.com/yan74/afplib
 - â─ Crawford Technologies - PRO AFP (crawfordtech.com)
 - â─ TargetStream Technologies - StreamEDS (targetstream.com)
 - â─ MPI Tech - AFP Fast Parser (mpitech.com)
 - â─ Apache FOP - AFP Renderer (xmlgraphics.apache.org/fop)
 - â─ IMARC Group - Mainframe Market Report 2024-2033
 - â─ MarketsandMarkets - Mainframe Modernization Market 2025-2030